

1. Lab Structure - VM / Docker / Process Boundaries

EMS VM

OPC UA Simulator
opcua_simulator.py
Creates State / Good / Reject
TCP 4840

OpenMES Host - WSL2 / Docker

DOCKER

OPC UA Gateway
Signal subscription + heartbeat

OpenMES Stack
Backend / API / DB - Work Order / Machine / Runtime

AWSQUICK-LAB VM

Shop-Floor App (app.py)
Work Order / Serial / PASS-FAIL / CSV

Mock Printer
mock_printer.py - TCP 9100

OPC UA :4840

REST API / HTTP

2. Normal Process - Confirm the normal flow first, then isolate faults

1 Create device data

EMS Simulator creates State / Good / Reject values and provides them through OPC UA.

2 Collect through Gateway

Docker Gateway subscribes to signals and sends readings + heartbeat to OpenMES.

3 Provide MES data

OpenMES provides Work Order, Machine Status, and Runtime status through REST APIs.

4 Run shop-floor flow

Serial scan -> PASS/FAIL -> CSV save -> label output through TCP 9100.

3. Three Fault Tests - Isolating different fault domains

FAULT 01 - DEVICE CONNECTIVITY

OPC UA Endpoint / Device Connection

Symptom

The Gateway cannot connect to the OPC UA endpoint and keeps retrying. Device data does not reach OpenMES.

old endpoint:
opc.tcp://192.168.40.128:4840

retry ...
connection failed

Check

Verified Simulator status -> IP -> TCP 4840 -> OpenMES connection setting.

Fix

Moved the Simulator to the EMS VM and changed the endpoint to 192.168.40.132:4840.

CHANGED GoodCount: 73
CHANGED GoodCount: 74
CHANGED GoodCount: 75

Verified: Gateway received live OPC UA values and changing counters again.

FAULT 02 - MIDDLEWARE / GATEWAY

Gateway Down / Heartbeat Lost

Test

Stopped the OPC UA Gateway on purpose and used the OpenMES runtime heartbeat to detect the communication failure.

docker stop openmes-opcua-gateway

Communication: HEARTBEAT LOST
State: UNKNOWN
Good: 0 (last known)
Reject: 0 (last known)

Decision

After more than 20 seconds without heartbeat, the app does not show old data as current. It shows UNKNOWN + last known.

Recovery

Restarted the Gateway and confirmed heartbeat returned.

Communication: HEALTHY
State: RUNNING

Verified: Normal -> Fault -> Detection -> Recovery cycle.

FAULT 03 - PRINTER / TCP

TCP Printer Connection Failure

Test

Processed a PASS result while the Mock Printer was not running.

Serial: SN-005
Result: PASS

Production record saved to CSV.
Printer simulation file generated: label.zpl
ERROR: Printer connection refused.

Isolation

Treated the MES transaction and Printer communication as separate fault domains.

Result

Work Order / Serial / PASS record and CSV save completed normally. Only the TCP 9100 print step failed.

Verified: Printer communication fault isolated from the MES transaction.

What This Lab Shows

This lab simulates a common manufacturing flow: work-order lookup, machine-status check, serial scan, PASS/FAIL recording, production-data logging, and label printing. Intentional faults were added across the Device -> Gateway -> MES -> Shop-Floor Terminal -> Printer path. I compared normal and fault conditions, isolated the fault location, and verified recovery. The project connects my telecom and network troubleshooting background to practical Manufacturing IT/OT support.

Technology

Linux - Docker - OpenMES - REST API - Laravel Sanctum - OPC UA - Python - TCP/IP

Artifacts

app.py - mock_printer.py - production_log.csv - label.zpl - README.md

Lab environment only. All values are test data, not production data.